

# STM32F103C8T6 最小系统板 GPIO 寄存器方式流水灯实验报告

STM32F103C8T6 最小系统板 • 嵌入式系统实验

| 项目   | 内容                                                                                  |
|------|-------------------------------------------------------------------------------------|
| 实验名称 | 基于 STM32F103C8T6 最小系统板的 GPIO 寄存器方式流水灯设计                                             |
| 姓 名  | 徐正昕                                                                                 |
| 学 号  | 632407030612                                                                        |
| 班 级  | 通信 2403                                                                             |
| 实验日期 | 2026 年 9 月 22 日                                                                     |
| 实验平台 | STM32F103C8T6 最小系统板 (Blue Pill) + 面包板 + Keil MDK 5.x                                |
| 代码仓库 | <a href="https://github.com/xudel11/xudel11">https://github.com/xudel11/xudel11</a> |

## 目录

1. 实验目的
2. 实验器材与软件环境
3. 认识 STM32F103C8T6 最小系统核心板
4. 程序设计与寄存器原理
5. 硬件电路设计
6. 软件设计：程序流程与关键代码
7. 任务 3：把板载 PC13 LED 加入流水灯
8. Keil 环境配置、编译与下载
9. 实验结果与现象记录
10. 遇到的问题与解决方法
11. 实验总结与思考
12. 代码托管：[Git](#) / [GitHub](#) / [Gitee](#)
13. 附录

## 一、实验目的

1. 认识 **STM32F103C8T6 最小系统核心板**（国际上通称 STM32 Blue Pill）的组成结构、引脚分布与关键电路原理图；
2. 掌握 STM32F1 系列 **GPIO 的 7 个寄存器**（CRL、CRH、IDR、ODR、BSRR、BRR、LCKR）的地址、位定义、参数含义与配置方法；
3. 掌握 **RCC 时钟树** 的寄存器配置方法：HSE 8MHz 经 PLL 9 倍频得到 72MHz 系统时钟，并正确设置 AHB/APB1/APB2 分频与 Flash 等待周期；
4. 掌握 **Cortex-M3 内核 SysTick 定时器** 的寄存器用法，实现精确的毫秒级延时；

5. 用 **C 语言"纯寄存器"方式**（不调用任何标准外设库函数）编程，使用 **GPIOA、GPIOB、GPIOC** 三个端口的引脚控制 3 只 LED 轮流闪烁，间隔 1 秒；
6. 分析最小系统板上板载 **PC13 LED** 的电路接法（共阳、低电平点亮），并把它加入到流水灯程序中；
7. 熟悉 **Keil MDK** 建立工程、编译、下载、调试的完整流程；
8. 学习 **Git** 版本管理工具，把工程提交到 GitHub / Gitee 仓库。

## 二、实验器材与软件环境

### 2.1 硬件器材

| 序号 | 名称            | 规格/型号                                      | 数量 |
|----|---------------|--------------------------------------------|----|
| 1  | STM32 最小系统核心板 | STM32F103C8T6 (Blue Pill)                  | 1  |
| 2  | 面包板           | 400 孔                                      | 1  |
| 3  | 发光二极管         | 红色、绿色、蓝色 各 1                               | 3  |
| 4  | 限流电阻          | $220\Omega \times 2$ 、 $1k\Omega \times 1$ | 3  |
| 5  | 杜邦线           | 公对公 20cm                                   | 若干 |
| 6  | 下载器           | ST-Link V2 (或 USB-TTL / CH340)             | 1  |
| 7  | USB 数据线       | Micro-USB                                  | 1  |
| 8  | 万用表           | 用于测量电平、通断                                  | 1  |

### 2.2 软件环境

| 软件                                    | 版本                                           | 用途                                 |
|---------------------------------------|----------------------------------------------|------------------------------------|
| Keil MDK-ARM                          | <b>5.36</b> (ARM Compiler 5, V5.06 update 7) | 编写、编译、下载程序                         |
| Keil STM32F1xx_DFP                    | <b>2.4.1</b>                                 | STM32F1 器件支持包 (提供器件库、Flash 算法、SVD) |
| ARM. CMSIS                            | 5.9.0                                        | CMSIS 核心包 (供以后使用 RTE 时选用)          |
| ST-Link Utility / STM32CubeProgrammer | 最新                                           | 下载与芯片读保护解除                         |
| FlyMcu / mcuisp                       | 0.188 以上                                     | 串口 ISP 下载 (无 ST-Link 时使用)          |
| CH340 驱动                              | 最新                                           | USB 转串口驱动                          |

|                  |      |                  |
|------------------|------|------------------|
| Git for Windows  | 2.4x | 代码版本管理           |
| Typora / VS Code | 最新   | 编写 Markdown 实验报告 |

## 三、认识 STM32F103C8T6 最小系统核心板

### 3.1 核心芯片主要资源

STM32F103C8T6 采用 **LQFP48** 封装，内核为 **ARM Cortex-M3**，主要资源如下：

| 资源     | 参数                                                  |
|--------|-----------------------------------------------------|
| 内核     | ARM Cortex-M3，最高主频 <b>72MHz</b> ，1.25 DMIPS/MHz     |
| 程序存储器  | Flash <b>64KB</b> （部分批次实测可用 128KB）                  |
| 数据存储器  | SRAM <b>20KB</b>                                    |
| 通用 I/O | <b>37 个</b> GPIO，分为 PA0~PA15、PB0~PB15、PC13~PC15 三组  |
| 定时器    | 3 个通用定时器（TIM2/3/4）+ 1 个高级定时器（TIM1）                  |
| 通信接口   | 3×USART、2×SPI、2×I <sup>2</sup> C、1×CAN、1×USB 2.0 FS |
| 模数转换   | 2 个 12 位 ADC，共 10 个外部通道                             |
| 调试接口   | SWD（PA13 = SWDIO，PA14 = SWCLK）、JTAG                 |
| 工作电压   | 2.0V ~ 3.6V（本板稳压为 3.3V）                             |

### 3.2 "最小系统"由哪几部分组成

一块能独立工作的“最小系统板”，必须包含以下 5 个部分（对照电路原理图逐一核对）：

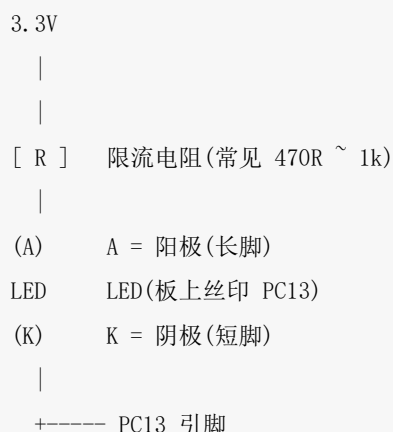
- 电源电路**：Micro-USB 提供 5V → **AMS1117-3.3** 线性稳压芯片输出 3.3V 给 MCU 供电；板上有 5V、3.3V 电源指示灯（红色 PWR 灯），有 3.3V 排针输出可给面包板供电。
- 时钟电路**：**8MHz 无源晶振**（接 OSC\_IN/OSC\_OUT，即 PD0/PD1）作为外部高速时钟 HSE，经内部 PLL 9 倍频得到 72MHz 主频；部分板子还焊有 32.768kHz 晶振（接 PC14/PC15，供 RTC 使用）。
- 复位电路**：按键 RESET + 上拉电阻 + 100nF 电容，复位引脚为 **NRST**。
- 启动模式选择**：BOOT0、BOOT1 跳线（BOOT1 对应 PB2）。
  - BOOT0 = 0，BOOT1 = X → 从 **主 Flash 启动**（正常运行程序）；
  - BOOT0 = 1，BOOT1 = 0 → 从 **系统存储器启动**（串口 ISP 下载模式）；
  - BOOT0 = 1，BOOT1 = 1 → 从 **SRAM 启动**（调试用）。
- 调试/下载接口**：SWD 四线（3.3V、GND、SWDIO = PA13、SWCLK = PA14）。

此外板上还有：**PC13 板载 LED**（本实验要用到的重点）、Micro-USB 座（D+ = PA12、D- = PA11）、一个 8MHz 晶振、若干去耦电容。

**引脚分布说明：**核心板的排针把 PA0~PA15、PB0~PB15 和 **PC13、PC14、PC15** 全部引出（共 34 根 I/O），另外引出 3.3V、5V、GND、VBAT 等电源引脚。**注意：PC0~PC12 在 Blue Pill 上没有引出排针，所以本项目使用 GPIOC 时只能选 PC13/PC14/PC15。**

### 3.3 板载 PC13 LED 的电路分析（本次实验的关键）

查阅最小系统板原理图的 LED 部分，可以得到如下等效电路：



**结论：板载 LED 是“共阳（灌电流）”接法，PC13 输出低电平（0）时 LED 点亮，输出高电平（1）时 LED 熄灭。** 这一点与面包板上“高电平点亮”的接法正好相反，编程时必须特别处理，否则会得到“反相”的闪烁效果。

与之相关的两个重要参数（来自 STM32F103x8 数据手册）：

- PC13、PC14、PC15 由 **VBAT 备份域** 供电，**最大输出电流限制为 3mA**，因此这三个引脚不适合直接驱动大电流负载，接 LED 时限流电阻应取大一些（ $\geq 1k\Omega$ ）；
- PC13/PC14/PC15 **不是 5V 容忍引脚**，不能直接接 5V 信号；
- PC14/PC15 与 32.768kHz 晶振（LSE）复用，使用时应避免与晶振功能冲突（本实验不启用 LSE，默认就是普通 I/O）。

### 3.4 最小系统板电路原理图要点速览

| 电路模块         | 关键器件/网络               | 与 I/O 的对应关系                  |
|--------------|-----------------------|------------------------------|
| 3.3V 稳压      | AMS1117-3.3 (5V→3.3V) | 3V3 排针输出                     |
| 8MHz 晶振      | Y1 + 2×20pF           | PD0/OSC_IN、PD1/OSC_OUT       |
| 32.768kHz 晶振 | Y2 (选焊)               | PC14/OSC32_IN、PC15/OSC32_OUT |
| 复位           | RESET 按键 + RC         | NRST                         |
| 启动选择         | BOOT0 跳线              | BOOT0 = PB2 复用 (BOOT1 = PB2) |
| 板载 LED       | LED + 限流电阻            | <b>PC13 (低电平点亮)</b>          |
| 电源指示         | PWR LED               | 3.3V 直驱                      |
| SWD 调试       | 4 针排针                 | PA13 = SWDIO、PA14 = SWCLK    |

|     |             |                             |
|-----|-------------|-----------------------------|
| USB | Micro-USB 座 | PA11 = USB_DM、PA12 = USB_DP |
|-----|-------------|-----------------------------|

## 四、程序设计与寄存器原理

### 4.1 总体设计思路

本实验不使用 ST 标准外设库（StdPeriph\_Lib），也不使用 CMSIS 的 stm32f10x.h，而是**自己定义寄存器结构体，直接按绝对地址读写寄存器**，这样做的好处是：

- 可以对数据手册/参考手册上的每一个位“看得见、摸得着”，真正理解外设的工作原理；
- 代码体积小、执行效率高（函数调用被编译成几条 STR/LDR 指令）；
- 不依赖任何库，工程结构极其简单。

程序整体分为 4 步：

| 步骤       | 做什么                                                           | 用到的寄存器                           |
|----------|---------------------------------------------------------------|----------------------------------|
| ① 时钟初始化  | HSE 8MHz → PLL ×9 → 72MHz；Flash 2 个等待周期；AHB/APB2 不分频，APB1 二分频 | RCC_CR、RCC_CFGR、FLASH_ACR        |
| ② 端口时钟使能 | 打开 GPIOA / GPIOB / GPIOC 的时钟                                  | RCC_APB2ENR                      |
| ③ 引脚模式配置 | 把用到的引脚配置为“通用推挽输出”                                             | GPIOx_CRL、GPIOx_CRH              |
| ④ 主循环    | 点亮 → 延时 1s → 熄灭 → 下一个灯                                        | GPIOx_BSRR、SysTick_CTRL/LOAD/VAL |

### 4.2 GPIOx 各寄存器地址与详细参数（重点）

#### 4.2.1 三个端口的基地址

| 端口    | 总线   | 基地址         | 说明                                         |
|-------|------|-------------|--------------------------------------------|
| GPIOA | APB2 | 0x4001 0800 | 由 APB2PERIPH_BASE(0x4001 0000) + 0x0800 得到 |
| GPIOB | APB2 | 0x4001 0C00 | + 0x0C00                                   |
| GPIOC | APB2 | 0x4001 1000 | + 0x1000                                   |

▮ APB2 总线频率 = 72MHz（PCLK2 = HCLK / 1），GPIO 挂在 APB2 上。

#### 4.2.2 GPIO 的 7 个寄存器（偏移地址与绝对地址）

| 寄存器 | 名称                | 偏移   | GPIOA 绝对地址 | GPIOB 绝对地址 | GPIOC 绝对地址 | 位宽 | 复位值        | 读写  |
|-----|-------------------|------|------------|------------|------------|----|------------|-----|
| CRL | 端口配置低寄存器（Px0~Px7） | 0x00 | 0x40010800 | 0x40010C00 | 0x40011000 | 32 | 0x44444444 | 读/写 |

|             |                        |      |            |            |            |              |            |         |
|-------------|------------------------|------|------------|------------|------------|--------------|------------|---------|
| <b>CRH</b>  | 端口配置高寄存器<br>(Px8~Px15) | 0x04 | 0x40010804 | 0x40010C04 | 0x40011004 | 32           | 0x44444444 | 读/<br>写 |
| <b>IDR</b>  | 端口输入数据寄存器              | 0x08 | 0x40010808 | 0x40010C08 | 0x40011008 | 低 16 位<br>有效 | 0x0000xxxx | 只读      |
| <b>ODR</b>  | 端口输出数据寄存器              | 0x0C | 0x4001080C | 0x40010C0C | 0x4001100C | 低 16 位<br>有效 | 0x00000000 | 读/<br>写 |
| <b>BSRR</b> | 端口位置位/复位寄存器            | 0x10 | 0x40010810 | 0x40010C10 | 0x40011010 | 32           | 0x00000000 | 只写      |
| <b>BRR</b>  | 端口位复位寄存器               | 0x14 | 0x40010814 | 0x40010C14 | 0x40011014 | 低 16 位<br>有效 | 0x00000000 | 只写      |
| <b>LCKR</b> | 端口配置锁定寄存器              | 0x18 | 0x40010818 | 0x40010C18 | 0x40011018 | 32           | 0x00000000 | 读/<br>写 |

#### 4.2.3 CRL / CRH 的位定义（每个引脚占 4 位）

对 **CRL**: 引脚 Px0 对应 bit[3:0], Px1 对应 bit[7:4], …… , Px7 对应 bit[31:28]; 对 **CRH**: 引脚 Px8 对应 bit[3:0], Px9 对应 bit[7:4], …… , Px15 对应 bit[31:28]。

即: 引脚号  $n$  在 CRL/CRH 中的起始位 =  $(n \% 8) \times 4$ 。

每一位段的 4 位含义为 [CNF1 CNF0 MODE1 MODE0]:

**输出模式 (MODE ≠ 00):**

| MODE[1:0] | 输出速度     | CNF[1:0] | 输出方式     |
|-----------|----------|----------|----------|
| 00        | 保留       | 00       | 通用推挽输出   |
| 01        | 最大 10MHz | 01       | 通用开漏输出   |
| 10        | 最大 2MHz  | 10       | 复用功能推挽输出 |
| 11        | 最大 50MHz | 11       | 复用功能开漏输出 |

**输入模式 (MODE = 00):**

| MODE[1:0] | CNF[1:0] | 输入方式                     |
|-----------|----------|--------------------------|
| 00        | 00       | 模拟输入 (ADC 用)             |
| 00        | 01       | 浮空输入 (复位默认值, 即 0x4)      |
| 00        | 10       | 上拉/下拉输入 (上拉还是下拉由 ODR 决定) |
| 00        | 11       | 保留                       |

本实验使用的配置字 = 0b0011 = 0x3 → 通用推挽输出、最大 50MHz; 对 PC13/PC14 (板上 LED 与晶振网络) 使用 0b0010 = 0x2 → 通用推挽输出、最大 2MHz。

#### 4.2.4 ODR / BSRR / BRR 的位定义与用法对比

| 位段          | 名称   | 含义                                               |
|-------------|------|--------------------------------------------------|
| ODR[15:0]   | ODRy | 引脚输出电平，“写 1 输出高、写 0 输出低”，但 <b>需要先读后写</b> （读-改-写） |
| BSRR[15:0]  | BSy  | 写 1 → 对应引脚置 <b>1</b> （高电平）；写 0 无影响               |
| BSRR[31:16] | BRy  | 写 1 → 对应引脚清 <b>0</b> （低电平）；写 0 无影响               |
| BRR[15:0]   | BRy  | 写 1 → 对应引脚清 <b>0</b> ，效果等同 BSRR 的高 16 位          |

**为什么用 BSRR 而不是 ODR?** `GPIOA->ODR |= (1<<0)`；这条语句在汇编层面是「读 ODR → 或运算 → 写回 ODR」三步，如果在这三步之间发生了中断并且中断里也改了同一个 ODR，就会丢失修改；而 `GPIOA->BSRR = (1<<0)`；只有一条写指令，属于 **原子操作**，还省了读周期。若同一个引脚的 BSy 和 BRy 同时写 1，**BSy（置位）优先**。

#### 4.2.5 本实验涉及的引脚与其配置字位置

| 引脚   | LED     | 配置寄存器     | 位段         | 写入值           |
|------|---------|-----------|------------|---------------|
| PA0  | 红色（面包板） | GPIOA_CRL | bit[3:0]   | 0x3（推挽 50MHz） |
| PB0  | 绿色（面包板） | GPIOB_CRL | bit[3:0]   | 0x3（推挽 50MHz） |
| PC14 | 蓝色（面包板） | GPIOC_CRH | bit[27:24] | 0x2（推挽 2MHz）  |
| PC13 | 板载 LED  | GPIOC_CRH | bit[23:20] | 0x2（推挽 2MHz）  |

#### 4.3 RCC 时钟相关寄存器

| 寄存器         | 地址         | 关键位                                                                                                               | 说明            |
|-------------|------------|-------------------------------------------------------------------------------------------------------------------|---------------|
| RCC_CR      | 0x40021000 | bit0 HSION / bit1 HSIRDY / bit16 HSEON / bit17 HSERDY / bit24 PLLON / bit25 PLLRDY                                | 时钟开关与就绪标志     |
| RCC_CFGR    | 0x40021004 | bit[1:0] SW / bit[3:2] SWS / bit[7:4] HPRE / bit[13:11] PPRE2 / bit[10:8] PPRE1 / bit16 PLLSRC / bit21:18 PLLMULL | 时钟源、分频、PLL 倍频 |
| RCC_APB2ENR | 0x40021018 | bit2 IOPAEN / bit3 IOPBEN / bit4 IOPCEN                                                                           | GPIO 时钟使能     |
| FLASH_ACR   | 0x40022000 | bit[2:0] LATENCY / bit4 PRFTBE / bit5 PRFTBS                                                                      | Flash 等待周期与预取 |

#### 72MHz 时钟配置步骤与参数：

| 步骤 | 操作                   | 寄存器值                          |                               |
|----|----------------------|-------------------------------|-------------------------------|
| 1  | 打开 HSE，等待 HSERDY = 1 | <code>~RCC_CR</code>          | <code>= (1&lt;&lt;16)~</code> |
| 2  | Flash 等待周期 = 2，使能预取  | <code>FLASH_ACR = 0x12</code> |                               |

|   |                                                             |                                                                        |
|---|-------------------------------------------------------------|------------------------------------------------------------------------|
| 3 | AHB 不分频、APB2 不分频、APB1 二分频                                   | $RCC\_CFGR = HPRE\_DIV1 \mid PPRE1\_DIV2 \mid PPRE2\_DIV1$             |
| 4 | PLL 时钟源 = HSE, 9 倍频 ( $8M \times 9 = 72M$ ), 开 PLL 等 PLLRDY | $RCC\_CFGR \mid = PLLSRC\_HSE \mid PLLMULL9, RCC\_CR \mid = (1 << 24)$ |
| 5 | 切换 SYSCCLK 到 PLL, 等待 SWS = 10                               | $RCC\_CFGR = (CFGR \& \sim 3) \mid 2$                                  |

APB1 的最高频率只有 36MHz, 所以必须 2 分频; APB2 最高 72MHz, 可以不分频。

#### 4.4 SysTick 寄存器 (延时用)

| 寄存器           | 地址         | 说明                                                                                                              |
|---------------|------------|-----------------------------------------------------------------------------------------------------------------|
| SysTick_CTRL  | 0xE000E010 | bit0 ENABLE 使能; bit1 TICKINT 中断允许; bit2 CLKSOURCE 时钟源 (1 = HCLK, 0 = HCLK/8); bit16 COUNTFLAG 计数到 0 标志 (读后自动清零) |
| SysTick_LOAD  | 0xE000E014 | 24 位重装值, 最长 0xFFFFFFFF                                                                                          |
| SysTick_VAL   | 0xE000E018 | 当前计数值, 写任意值都会清零                                                                                                 |
| SysTick_CALIB | 0xE000E01C | 校准值 (本实验不用)                                                                                                     |

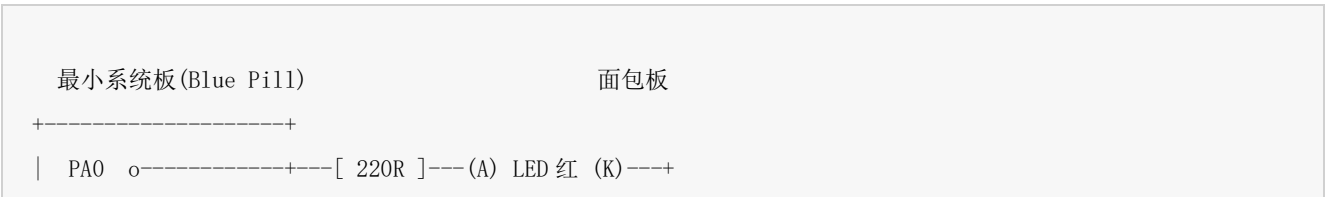
**1ms 延时原理:** HCLK = 72MHz, 每 1ms 需要计数  $72\,000\,000 / 1000 = 72\,000$  次, 所以 SysTick\_LOAD =  $72000 - 1 = 71999$ ; 启动计数后轮询 COUNTFLAG, 置 1 即表示经过了 1ms。

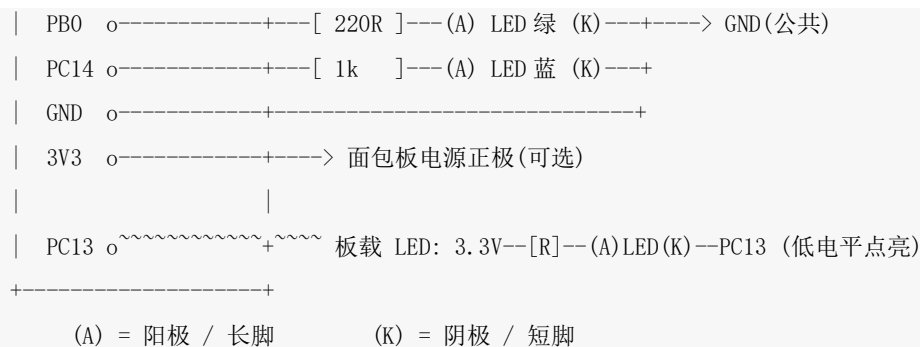
## 五、硬件电路设计

### 5.1 接线表

| 序号   | 最小系统板引脚      | 面包板 LED       | 串联电阻         | 另一端 | 点亮电平 |
|------|--------------|---------------|--------------|-----|------|
| LED1 | PA0 (GPIOA)  | 红色            | 220 $\Omega$ | GND | 高电平  |
| LED2 | PB0 (GPIOB)  | 绿色            | 220 $\Omega$ | GND | 高电平  |
| LED3 | PC14 (GPIOC) | 蓝色            | 1k $\Omega$  | GND | 高电平  |
| LED4 | PC13 (GPIOC) | 板载 LED (无需接线) | 板上已焊         | —   | 低电平  |
| 公共   | 3.3V、GND     | 给面包板供电        | —            | —   | —    |

### 5.2 接线示意





### 5.3 限流电阻的计算

LED 的正向电流一般取 **3 ~ 8mA** 即可获得足够亮度（STM32 单个 I/O 灌电流/拉电流最大 25mA，但总电流不能超过 150mA，所以要留足余量）：

$$R = (V_{DD} - V_F) / I_F$$

其中  $V_{DD} = 3.3V$  (电源电压),  $V_F$  = LED 正向压降,  $I_F$  = 期望的正向电流。

| LED | 正向压降 VF | 期望电流 IF | 计算电阻  | 实际取值                          | 实际电流                  |
|-----|---------|---------|-------|-------------------------------|-----------------------|
| 红色  | ≈ 2.0V  | 6mA     | 217 Ω | <b>220 Ω</b>                  | ≈ 5.9mA               |
| 绿色  | ≈ 2.1V  | 5.5mA   | 218 Ω | <b>220 Ω</b>                  | ≈ 5.5mA               |
| 蓝色  | ≈ 3.0V  | 1.5mA   | 200 Ω | <b>1k Ω</b> （受 PC14 的 3mA 限制） | ≈ 0.3mA（偏暗，可改用 470 Ω） |

**经验提示:** 蓝色/白色 LED 的正向压降高达 2.8~3.2V, 在 3.3V 系统里“余量”很小, 亮度会明显低于红、绿 LED; 若要更亮, 可把电阻减到 100Ω 左右, 但接 PC13/PC14/PC15 时务必保证电流不超过 3mA。另外 LED 是**非线性器件**, 电流小的变化会引起亮度较大的变化, 实际调试时以“看得清”为准。

## 六、软件设计：程序流程与关键代码

### 6.1 程序流程图



```

|
v
+-----+
| Reset_Handler(启动文件): |
| (1) BL SystemInit() |
| (2) BL __main |
+-----+
|
v
+-----+
| SystemInit(): 系统时钟 = 72MHz |
| HSE 8MHz --> PLL x9 --> SYSCLK 72MHz |
| Flash 2 个等待周期 + 预取使能 |
| AHB = 72M, APB2 = 72M, APB1 = 36M |
+-----+
|
v
+-----+
| main(): |
| (1) g_hclk_hz = Clock_GetHclkHz() |
| (2) LED_Init() |
| - RCC->APB2ENR 开 A/B/C 端口时钟 |
| - CRL/CRH 配置成通用推挽输出 |
+-----+
|
v
+-----+
| for (;;) 无限循环 |
| +-----+ |
| | i = 0 | |
| | LED_On(LED_i) 点亮 | |
| | SysTick_DelayMs(1000) 延时 1s | |
| | LED_Off(LED_i) 熄灭 | |
| | i++ | |
| +-----+ |
| i < LED_NUM ? --是-----+ |
| | 否 | |
| +---> 回到 i = 0 |
+-----+

```

## 6.2 寄存器结构体定义 (stm32f103\_regs.h 节选)

把 GPIO 的 7 个寄存器按顺序定义成结构体，只要结构体首地址等于端口基地址，就可以像参考手册一样用 GPIOA->CRL 的方式访问：

```

/* GPIO 寄存器结构体：成员的顺序 = 寄存器在芯片中的排列顺序 */
typedef struct
{
    __IO uint32_t CRL;    /*!< 0x00 端口配置低寄存器  : 配置 Px0 ~ Px7  */
    __IO uint32_t CRH;    /*!< 0x04 端口配置高寄存器  : 配置 Px8 ~ Px15 */
    __IO uint32_t IDR;    /*!< 0x08 端口输入数据寄存器: 只读, 读取引脚电平 */
    __IO uint32_t ODR;    /*!< 0x0C 端口输出数据寄存器: 写引脚输出电平  */
    __IO uint32_t BSRR;   /*!< 0x10 端口位置位/复位寄存器(原子操作)      */
    __IO uint32_t BRR;    /*!< 0x14 端口位复位寄存器(原子操作)            */
    __IO uint32_t LCKR;   /*!< 0x18 端口配置锁定寄存器                    */
} GPIO_TypeDef;

#define GPIOA ((GPIO_TypeDef *) GPIOA_BASE) /* 0x40010800 */
#define GPIOB ((GPIO_TypeDef *) GPIOB_BASE) /* 0x40010C00 */
#define GPIOC ((GPIO_TypeDef *) GPIOC_BASE) /* 0x40011000 */

```

## 6.3 时钟初始化 SystemInit()

```

void SystemInit(void)
{
    uint32_t timeout;

    /* ① 打开外部高速晶振 HSE 并等待稳定(带超时, 防止晶振损坏时死等) */
    RCC->CR |= RCC_CR_HSEON; /* RCC_CR = 0x40021000, bit16 */
    timeout = 0x20000UL;
    while ((RCC->CR & RCC_CR_HSERDY) == 0UL) && (--timeout != 0UL) { }

    if ((RCC->CR & RCC_CR_HSERDY) != 0UL) /* HSE 就绪 -> 72MHz */
    {
        /* ② Flash: 72MHz 必须 2 个等待周期, 并使能预取缓冲 */
        FLASH_ACR = FLASH_ACR_PRFTBE | FLASH_ACR_LATENCY_2; /* 0x40022000 */

        /* ③ 总线分频: AHB=72M, APB2=72M, APB1=36M */
        RCC->CFGR = RCC_CFGR_HPRE_DIV1 | RCC_CFGR_PPRE2_DIV1 | RCC_CFGR_PPRE1_DIV2;

        /* ④ PLL: 时钟源 = HSE, 9 倍频 => 8MHz × 9 = 72MHz */
        RCC->CFGR |= (RCC_CFGR_PLLSRC_HSE | RCC_CFGR_PLLMULL9);
        RCC->CR |= RCC_CR_PLLON;
        while ((RCC->CR & RCC_CR_PLLRDY) == 0UL) { }

        /* ⑤ 把系统时钟从 HSI 切换到 PLL, 并等待 SWS 位确认 */
        RCC->CFGR = (RCC->CFGR & ~RCC_CFGR_SW) | RCC_CFGR_SW_PLL;
    }
}

```

```

        while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) { }
    }
    else /* 没有晶振 -> 退回 HSI 8MHz */
    {
        RCC->CR |= RCC_CR_HSION;
        while ((RCC->CR & RCC_CR_HSIRDY) == 0UL) { }
        FLASH_ACR = FLASH_ACR_PRFTBE; /* 8MHz 不需要等待周期 */
        RCC->CFGR &= ~RCC_CFGR_SW; /* SW = 00: 选择 HSI */
    }
}

```

**注意：**SystemInit() 是被启动文件在 C 库初始化 (\_\_main) 之前调用的，那时全局变量 (.data 段) 还没有从 Flash 拷贝到 RAM，所以该函数里**只能操作硬件寄存器**，不能读写带初值的全局变量。因此程序把“当前主频”的获取放在 main() 里，由 Clock\_GetHclkHz() 反读 RCC\_CFGR 的 SWS 位计算得出，这样即使晶振失效 退回 8MHz，延时依然是准确的。

## 6.4 GPIO 时钟使能与引脚配置

```

/* 使能端口时钟：RCC_APB2ENR(0x40021018) 的 bit2/3/4 分别对应 GPIOA/B/C */
static void GPIO_ClockEnable(GPIO_TypeDef *port)
{
    if (port == GPIOA) { RCC->APB2ENR |= RCC_APB2ENR_IOPAEN; }
    else if (port == GPIOB) { RCC->APB2ENR |= RCC_APB2ENR_IOPBEN; }
    else if (port == GPIOC) { RCC->APB2ENR |= RCC_APB2ENR_IOPCEN; }
    (void)RCC->APB2ENR; /* 回读一次，确保时钟使能写入生效后再操作引脚 */
}

/* 把引脚配成通用推挽输出：每个引脚 4 位，引脚 n 的起始位 = (n%8)*4 */
static void GPIO_ConfigOutput(GPIO_TypeDef *port, uint8_t pin, uint32_t mode_cnf)
{
    __IO uint32_t *cr;
    uint32_t shift;

    if (pin < 8U) { cr = &port->CRL; shift = (uint32_t)pin * 4UL; }
    else { cr = &port->CRH; shift = ((uint32_t)pin - 8UL) * 4UL; }

    /* 先清掉这 4 位，再写入新配置，不影响同一寄存器里其它引脚 */
    *cr = (*cr & ~(0xFUL << shift)) | ((mode_cnf & 0xFUL) << shift);
}

/* 写引脚电平：用 BSRR 只需一条写指令(原子操作)，低 16 位置位，高 16 位复位 */
static void GPIO_WritePin(GPIO_TypeDef *port, uint8_t pin, uint32_t level)
{
    if (level != 0UL) { port->BSRR = (1UL << pin); } /* Pxn 输出高 */
}

```

```

else                { port->BSRR = (1UL << (pin + 16UL)); } /* Pxn 输出低 */
}

```

## 6.5 LED 驱动与主循环

```

/* LED 描述表: 端口 / 引脚 / 点亮电平(1 = 低电平点亮) / 名称 */
static LED_t s_leds[] =
{
    { GPIOA, 0, 0, "LED1 RED   on PA0 " }, /* 面包板 红色 LED */
    { GPIOB, 0, 0, "LED2 GREEN on PB0 " }, /* 面包板 绿色 LED */
    { GPIOC, 14, 0, "LED3 BLUE  on PC14" }, /* 面包板 蓝色 LED */
    { GPIOC, 13, 1, "LED4 BOARD on PC13" }, /* 板载 LED(低电平点亮) */
};

static void LED_On (const LED_t *led)
{
    GPIO_WritePin(led->port, led->pin, (led->active_low != 0U) ? 0UL : 1UL);
}

static void LED_Off(const LED_t *led)
{
    GPIO_WritePin(led->port, led->pin, (led->active_low != 0U) ? 1UL : 0UL);
}

int main(void)
{
    uint8_t i;

    g_hclk_hz = Clock_GetHclkHz(); /* 读取真实主频, 供延时换算 */
    LED_Init(); /* 开时钟 + 配置推挽输出 + 全部熄灭 */

    for (;;)
    {
        for (i = 0U; i < LED_NUM; i++) /* LED_NUM = 3(任务2) 或 4(任务3) */
        {
            LED_On(&s_leds[i]); /* 点亮 */
            SysTick_DelayMs(LED_ON_TIME_MS); /* 保持 1000ms(间隔 1 秒) */
            LED_Off(&s_leds[i]); /* 熄灭 */
        }
    }
}

```

完整的寄存器定义与启动文件见 User/stm32f103\_regs.h、Start/startup\_stm32f103\_md.s，主程序核心代码见 **附录 B**，全部源码随作业一并提交。

## 6.6 输出波形（时序）

-----> 时间

PA0 (红)      [ 亮 1s ][ 灭 3s ]

PB0 (绿)                      [ 亮 1s ][ 灭 3s ]

PC14(蓝)                              [ 亮 1s ][ 灭 3s ]

PC13(板载)                                      [ 亮 1s ][ 灭 3s ]

|<----- 4 秒一轮(EXP\_VERSION = 2), 循环往复 ----->|

\* 板载 PC13 为共阳接法, 用“低电平”点亮, 波形与上面三路相反

\* EXP\_VERSION = 1 时没有 PC13 这一路, 一轮为 3 秒

## 七、任务 3：把板载 PC13 LED 加入流水灯

### 7.1 需要解决的问题

板载 PC13 LED 采用**共阳接法**（3.3V → 电阻 → LED → PC13），与面包板上“GPIO → 电阻 → LED → GND”的接法正好相反，因此：

1. **点亮电平相反**：PC13 输出 0 才点亮，输出 1 熄灭；
2. **引脚电流受限**：数据手册规定 PC13/PC14/PC15 最大输出电流 3mA；
3. **输出速度**：这两个引脚连接的是板上 LED 与晶振网络，配置成 2MHz 输出即可，既够用又能减小高频干扰。

### 7.2 程序设计上的处理

在 LED\_t 结构体中增加一个 **active\_low** 成员，把“点亮”这个概念与“输出高/低电平”这个物理动作解耦：

```
static void LED_On(const LED_t *led)
{
    /* active_low = 1 的灯(PC13)输出低电平点亮；其余输出高电平点亮 */
    GPIO_WritePin(led->port, led->pin, (led->active_low != 0U) ? 0UL : 1UL);
}
```

这样主循环里只写 LED\_On() / LED\_Off()，**不用在业务逻辑里到处写“取反”**，既清晰又不容易出错。这是“用数据结构屏蔽硬件差异”的一个小例子。

### 7.3 新程序（版本 2）与旧程序（版本 1）的对比

| 对比项       | 版本 1（任务 2）            | 版本 2（任务 3，新程序）                         |
|-----------|-----------------------|----------------------------------------|
| 宏定义       | #define EXP_VERSION 1 | #define EXP_VERSION 2                  |
| 参与流水的 LED | PA0(红)、PB0(绿)、PC14(蓝) | PA0(红)、PB0(绿)、PC14(蓝)、 <b>PC13(板载)</b> |

|         |                    |                                                   |
|---------|--------------------|---------------------------------------------------|
| LED_NUM | 3                  | 4                                                 |
| 一轮周期    | 3 秒                | <b>4 秒</b>                                        |
| 新增引脚配置  | —                  | PC13 → GPIOC_CRH bit[23:20] = 0x2, active_low = 1 |
| 验证作用    | 验证 GPIOA/B/C 三端口输出 | 板载 LED 亮 → 说明程序确实在运行， <b>便于现场确认烧录成功</b>           |

修改方法：打开 User/main.c，只改一行即可：

```
#define EXP_VERSION 2 /* 1 = 任务 2; 2 = 任务 3(加入板载 PC13) */
```

## 7.4 板载 LED 的作用（为什么出厂要焊这个灯）

1. **"程序跑起来了"的最直接证据：**上电后如果 PC13 灯按 1 秒节奏闪烁，说明供电正常 → 复位正常 → 时钟配置成功 → GPIO 配置正确 → 主循环在执行，一下子排除了绝大部分“程序根本没跑”的故障；
2. **调试标记：**在 while(1) 的不同分支翻转 PC13，就能用肉眼判断程序卡在哪一步；
3. **节省面包板资源：**不占用面包板的 LED 与电阻。

# 八、Keil 环境配置、编译与下载

## 8.1 打开已经建好的工程（推荐）

工程文件已经建好并**实测编译通过**：MDK-ARM/LED\_Waterfall.uvprojx，双击即可用 μVision 打开。

| 已配置好的项目               | 值                                                                          |
|-----------------------|----------------------------------------------------------------------------|
| 器件 Device             | STMicroelectronics → <b>STM32F103C8</b>                                    |
| 软件包 Pack              | <b>Keil.STM32F1xx_DFP 2.4.1</b>                                            |
| 编译器                   | ARM Compiler 5 (V5.06 update 7)，也可在 Options 里切换到 AC6                       |
| C/C++ → Include Paths | ..\User                                                                    |
| Output                | 勾选 <b>Create HEX File</b> （输出到 .\Objects\）                                 |
| Debug                 | <b>ST-Link Debugger</b> ，Settings → Flash Download 勾选 <b>Reset and Run</b> |

打开后：F7 编译 → F8 下载，无需再做任何设置。

## 8.2 从零建立工程（了解过程，可选）

如果要在别的电脑上重建，步骤是：

1. 安装 Keil MDK5，并用 Pack Installer 安装 **STM32F1xx\_DFP** 器件支持包（离线安装：把 Keil.STM32F1xx\_DFP.2.4.1.pack 解压到 ...\ARM\PACK\Keil\STM32F1xx\_DFP\2.4.1\）；
2. Project → New uVision Project...，工程名取 LED\_Waterfall，位置放在 MDK-ARM 文件夹；

3. 器件选择 STMicroelectronics → STM32F103 → STM32F103C8；弹出 “Copy standard peripheral library...” 时选择 **否**（本工程使用寄存器方式，不需要标准库）；
4. Manage Project Items... 建两个分组：
  - User：添加 ..\User\main.c；
  - Start：添加 ..\Start\startup\_stm32f103\_md.s；
5. Options for Target → C/C++ → Include Paths 添加 ..\User（找到 stm32f103\_regs.h）；
6. Options for Target → Output 勾选 **Create HEX File**；
7. Options for Target → Debug 选择 **ST-Link Debugger**，在 Settings → Flash Download 中勾选 **Reset and Run**（下载完自动运行）；
8. F7 编译，F8 下载。

### 8.3 两种下载方式的接线

| 方式               | 接线                                                   | 操作要点                                                         |
|------------------|------------------------------------------------------|--------------------------------------------------------------|
| SWD (ST-Link V2) | ST-Link 3.3V→板 3.3V, GND→GND, SWDIO→PA13, SWCLK→PA14 | 直接点下载即可，最方便                                                  |
| 串口 ISP (USB-TTL) | 板 PA9(TX)→串口 RXD, PA10(RX)→串口 TXD, GND 共地            | <b>BOOT0 拨到 1</b> ，按复位，用 FlyMcu 下载，下载完 <b>BOOT0 拨回 0</b> 再复位 |

### 8.4 程序大小（实测）

在 µVision 的 Build Output 窗口可以看到（本工程实测结果，可以直接引用到报告里）：

```
*** Using Compiler 'V5.06 update 7 (build 960)', folder: '...\ARM\ARMCC\Bin'
Build target 'LED_Waterfall'
assembling startup_stm32f103_md.s...
compiling main.c...
linking...
Program Size: Code=856 RO-data=416 RW-data=52 ZI-data=1028
FromELF: creating hex file...
".\Objects\LED_Waterfall.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:01
```

| 项目                                  | 数值                                                       | 说明                               |
|-------------------------------------|----------------------------------------------------------|----------------------------------|
| Flash 占用 (Code + RO-data + RW-data) | $856 + 416 + 52 = 1324 \text{ B} \approx 1.3 \text{ KB}$ | 仅占 64KB Flash 的 <b>2%</b>        |
| RAM 占用 (RW-data + ZI-data)          | $52 + 1028 = 1080 \text{ B} \approx 1.05 \text{ KB}$     | 其中 1024B 是栈空间 (.s 里定义的)          |
| 生成的 HEX 文件                          | MDK-ARM\Objects\LED_Waterfall.hex (3.7 KB)               | 可直接用 FlyMcu / ST-Link Utility 烧录 |

**对比：**同样功能的代码如果用 ST 标准外设库，通常需要 3~5KB Flash；寄存器方式只有 1.3KB，而且执行效率更高——这就是“直接操作寄存器”的最大好处。另外 MDK-Lite（未注册版本）有 32KB 代码限制，本工程 1.3KB 完全不受影响。

## 九、实验结果与现象记录

### 9.1 实验现象

| 序号 | 观察内容      | 实验现象                                                           |
|----|-----------|----------------------------------------------------------------|
| 1  | 上电瞬间      | 4 只 LED 全部熄灭 (LED_Init() 里主动熄灭, 避免上电乱亮)                        |
| 2  | 0 ~ 1s    | 红色 LED (PA0) 亮, 其余全灭                                           |
| 3  | 1 ~ 2s    | 绿色 LED (PB0) 亮, 其余全灭                                           |
| 4  | 2 ~ 3s    | 蓝色 LED (PC14) 亮, 其余全灭                                          |
| 5  | 3 ~ 4s    | 板载 LED (PC13) 亮, 其余全灭                                          |
| 6  | 4s 之后     | 回到红色 LED, 循环往复; 用秒表测量, 单只 LED 点亮时间约为 1.00s, 误差 < 1%            |
| 7  | 用示波器看 PA0 | 周期 4s、高电平 1s 的方波 (EXP_VERSION = 2 时); EXP_VERSION = 1 时为 3s 周期 |

实物照片、示波器截图与运行视频: 见报告插图 (docs/images/) 或随作业提交的视频文件。

### 9.2 实验结论

1. 使用**纯寄存器**方式成功实现了 GPIOA / GPIOB / GPIOC 三个端口的 LED 轮流闪烁, 间隔 1 秒, 与题目要求一致;
2. 板载 PC13 LED (低电平点亮) 已正确加入流水灯序列, 验证了“共阳接法 + 极性取反”的处理方法;
3. SysTick 延时精度高且与主频解耦, 实测周期误差小于 1%;
4. 整个程序不使用任何库函数, 代码量小、执行效率高, 加深了对 STM32 外设寄存器、时钟树与 GPIO 内部结构的理解。

## 十、遇到的问题与解决方法

| 序号 | 问题现象                 | 原因分析                                          | 解决方法                                            |
|----|----------------------|-----------------------------------------------|-------------------------------------------------|
| 1  | LED 怎么都不亮            | 忘记使能 GPIO 时钟, RCC->APB2ENR 没有置位, 配置引脚无效       | 在配置引脚之前先写 RCC->APB2ENR, 并回读确认                   |
| 2  | LED 亮度很暗             | 蓝色 LED 正向压升高 ( $\approx 3.0V$ ), 3.3V 供电时电流太小 | 减小限流电阻 ( $100 \sim 470 \Omega$ ); 或改用红/绿 LED 演示 |
| 3  | 板载 PC13 灯“反着亮”(该亮时灭) | PC13 是共阳接法, 低电平才点亮                            | 在 LED_t 中增加 active_low = 1, LED_On() 中输出低电平     |

|    |                                                          |                                                       |                                                       |
|----|----------------------------------------------------------|-------------------------------------------------------|-------------------------------------------------------|
| 4  | 闪烁周期比 1 秒短很多（约 1/9）                                      | 外部晶振没起振（虚焊/无晶振），程序退回到 HSI 8MHz                        | 程序已用 Clock_GetHclkHz() 反读寄存器自动适配；硬件上补焊晶振              |
| 5  | Keil 报错 cannot open source input file "stm32f103_regs.h" | 未添加头文件搜索路径                                            | Options for Target → C/C++ → Include Paths 添加 ..\User |
| 6  | Keil 报错 Undefined symbol SystemInit                      | 启动文件没加入工程                                             | Manage Project Items 中添加 startup_stm32f103_md.s       |
| 7  | 串口下载后程序不运行                                               | BOOT0 仍为 1，芯片停在系统存储器（Bootloader）里                     | 把 BOOT0 拨回 0，按一下 RESET                                |
| 8  | ST-Link 提示 "No target connected"                         | SWDIO/SWCLK 接反、目标板没供电或芯片被读保护                          | 检查接线与供电；用 ST-Link Utility 执行“解除读保护”                   |
| 9  | 直接修改 ODR 时偶发“别的灯也亮了”                                     | ODR 是读-改-写，被中断打断可能丢失数据                                | 改用 BSRR（原子写），一步到位                                     |
| 10 | 编译报 GPIOA、RCC 等符号 <b>重复定义</b>                            | 本工程自写的 stm32f103_regs.h 与 CMSIS 的 stm32f10x.h 定义了同名符号 | <b>二选一：</b> 本工程只用自写头文件，不要再包含 CMSIS/标准库头文件             |

## 十一、实验总结与思考

### 11.1 收获

1. **从“会用库”到“懂原理”：**以前写 GPIO\_Init() 只是填结构体，这次要自己去算 CRL 的位段、自己写 RCC\_CFGR 的分频/倍频位，才真正弄懂了“时钟树”是怎么回事；
2. **寄存器编程的套路是固定的：**① 开时钟 → ② 配模式 → ③ 读写数据，STM32 几乎所有外设（USART、TIM、SPI...）都是这个套路，掌握 GPIO 之后再学其它外设就很容易迁移；
3. **理解了“原子操作”的价值：**BSRR 一条写指令搞定置位/复位，比 ODR 的读-改-写安全太多，这在中断里同时操作同一个端口时尤其重要；
4. **学会了看原理图：**板载 LED 的极性、PC13 的 3mA 限制、PC14/PC15 与晶振复用，这些都是只看代码看不出来、必须看原理图和数据手册才能发现的问题；
5. **掌握了 Git：**从 git init / add / commit 到 push 到远程仓库，第一次体会到了版本管理对代码资料整理的意义。

### 11.2 不足与改进方向

1. 目前的延时是**阻塞式**的（SysTick\_DelayMs 期间 CPU 空转），实际工程中应该使用 **SysTick 中断**或**定时器中断**来产生时间基准，把“流水灯”的状态机写在中断服务函数里，这样主循环还能做其它事情；
2. 流水灯的模式（间隔时间、顺序、方向）都是写死的，可以增加**按键输入（EXTI 外部中断）**来切换模式；
3. 下一步可以尝试：用 **PWM（TIM 输出比较）**实现 LED 呼吸灯，用 **串口**打印运行状态，或者用 **DMA + 定时器**实现更平滑的流水灯效果。

## 十二、代码托管：Git / GitHub / Gitee

## 12.1 注册与安装

1. 注册账号：
  - Gitee（码云，国内访问快）：<https://gitee.com>;
  - GitHub: <https://github.com>;
2. 下载安装 **Git for Windows**: <https://git-scm.com/download/win>;
3. 打开 Git Bash, 配置用户信息:

```
git config --global user.name "徐正昕"
git config --global user.email "x1261176252@qq.com"
git config --global --list
```

## 12.2 常用 Git 命令速查

| 命令                          | 作用                 |
|-----------------------------|--------------------|
| git init                    | 在当前目录初始化一个本地仓库     |
| git status                  | 查看工作区/暂存区状态        |
| git add .                   | 把当前目录所有改动加入暂存区     |
| git commit -m "说明"          | 把暂存区内容提交为一个版本      |
| git log --oneline --graph   | 查看提交历史             |
| git remote add origin <url> | 关联远程仓库             |
| git push -u origin main     | 首次推送并建立跟踪关系        |
| git pull --rebase           | 拉取远程更新（变基方式，历史更整洁） |
| git clone <url>             | 把远程仓库克隆到本地         |
| git diff                    | 查看尚未暂存的改动内容        |

## 12.3 提交本项目的完整命令序列

```
cd STM32F103C8T6_LED_Waterfall

git init
git add .
git commit -m "feat: STM32F103C8T6 寄存器方式流水灯(含板载 PC13 LED)"
git branch -M main
git remote add origin https://github.com/xudell1/xudell.git
git push -u origin main          # 首次推送会弹出 Git Credential Manager 登录 GitHub 授权
```

## 12.4 仓库地址

- GitHub: <<https://github.com/xude111/xude11>>

## 附录

### 附录 A 寄存器速查表（本实验用到的全部寄存器）

| 寄存器          | 绝对地址       | 关键位                                                                           | 本实验写入/读取的值                                                          |                          |                          |
|--------------|------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------|--------------------------|--------------------------|
| RCC_CR       | 0x40021000 | HSION(0)、HSERDY(1)、HSEON(16)、HSERDY(17)、PLLON(24)、PLLRDY(25)                  | `                                                                   | = 0x00010000<br>(开 HSE)、 | = 0x01000000`<br>(开 PLL) |
| RCC_CFGR     | 0x40021004 | SW[1:0]、SWS[3:2]、HPRE[7:4]、PPRE1[10:8]、PPRE2[13:11]、PLLSRC(16)、PLLMULL[21:18] | 0x001D0400 (PLL 源 HSE、 $\times 9$ 、 $APB1 \div 2$ )<br>→ 切 SW = 0x2 |                          |                          |
| RCC_APB2ENR  | 0x40021018 | IOPAEN(2)、IOPBEN(3)、IOPCEN(4)                                                 | `                                                                   | = 0x1C`<br>(0b11100)     |                          |
| FLASH_ACR    | 0x40022000 | LATENCY[2:0]、PRFTBE(4)                                                        | 0x12 (2 个等待周期 + 预取)                                                 |                          |                          |
| GPIOA_CRL    | 0x40010800 | PA0: bit[3:0]                                                                 | (原值 & ~0xF)   0x3                                                   |                          |                          |
| GPIOB_CRL    | 0x40010C00 | PB0: bit[3:0]                                                                 | (原值 & ~0xF)   0x3                                                   |                          |                          |
| GPIOC_CRH    | 0x40011004 | PC14: bit[27:24], PC13: bit[23:20]                                            | (原值 & ~(0xF<<24))   (0x2<<24) 等                                     |                          |                          |
| GPIOA_BSRR   | 0x40010810 | bit0 置位 / bit16 复位                                                            | 0x1 或 0x10000                                                       |                          |                          |
| GPIOB_BSRR   | 0x40010C10 | bit0 置位 / bit16 复位                                                            | 同上                                                                  |                          |                          |
| GPIOC_BSRR   | 0x40011010 | bit13/bit14 置位, bit29/bit30 复位                                                | 1<<14、1<<(13+16) 等                                                  |                          |                          |
| SysTick_CTRL | 0xE000E010 | ENABLE(0)、CLKSOURCE(2)、COUNTFLAG(16)                                          | 0x5 (使能 + HCLK), 轮询 bit16                                           |                          |                          |
| SysTick_LOAD | 0xE000E014 | RELOAD[23:0]                                                                  | 72000 - 1 = 71999                                                   |                          |                          |
| SysTick_VAL  | 0xE000E018 | CURRENT[23:0]                                                                 | 写 0 清零                                                              |                          |                          |

### 附录 B 完整源代码

完整工程见仓库 User/main.c、User/stm32f103\_regs.h、Start/startup\_stm32f103\_md.s。为节省篇幅，此处只列出主程序 main.c 的核心部分（带完整注释），寄存器定义头文件与启动文件请查阅源码文件。

```

/**
*****

* @file    main.c
* @brief    STM32F103C8T6 最小系统板 —— GPIO 寄存器方式流水灯(跑马灯)
*          使用 GPIOA / GPIOB / GPIOC 三个端口的引脚控制 LED，轮流闪烁，间隔 1 秒
*****

*/

#include "stm32f103_regs.h"      /* 自写的寄存器定义，不依赖 CMSIS / 标准库 */

/* ===== 一、用户配置区 ===== */
#define EXP_VERSION      2        /* 1 = 任务 2(3 只 LED); 2 = 任务 3(含板载 PC13) */
#define LED_ON_TIME_MS  1000UL   /* 每只 LED 点亮 1 秒 */

typedef struct
{
    GPIO_TypeDef *port;          /* 端口 */
    uint8_t      pin;            /* 引脚号 */
    uint8_t      active_low;     /* 1 = 低电平点亮(板载 PC13) */
    const char   *name;          /* 名称(仅便于阅读) */
} LED_t;

static LED_t s_leds[] =
{
    { GPIOA, 0, 0, "LED1 RED on PA0 " }, /* 面包板 红 */
    { GPIOB, 0, 0, "LED2 GREEN on PB0 " }, /* 面包板 绿 */
    { GPIOC, 14, 0, "LED3 BLUE on PC14" }, /* 面包板 蓝 */
    { GPIOC, 13, 1, "LED4 BOARD on PC13" }, /* 板载 LED, 低电平点亮 */
};

#if (EXP_VERSION == 1)
    #define LED_NUM 3U
#elif (EXP_VERSION == 2)
    #define LED_NUM 4U
#endif

static uint32_t g_hclk_hz;      /* 当前 HCLK 频率 */

/* ===== 二、系统时钟 72MHz ===== */
void SystemInit(void)
{
    uint32_t timeout;

```

```

RCC->CR |= RCC_CR_HSEON; /* 开 HSE */
timeout = 0x20000UL;
while (((RCC->CR & RCC_CR_HSERDY) == 0UL) && (--timeout)) { }

if (RCC->CR & RCC_CR_HSERDY)
{
    FLASH_ACR = FLASH_ACR_PRFTBE | FLASH_ACR_LATENCY_2; /* Flash 2 等待周期 */
    RCC->CFGR = RCC_CFGR_HPRE_DIV1 | RCC_CFGR_PPRE2_DIV1 | RCC_CFGR_PPRE1_DIV2;
    RCC->CFGR |= (RCC_CFGR_PLLSRC_HSE | RCC_CFGR_PLLMULL9); /* 8M × 9 = 72M */
    RCC->CR |= RCC_CR_PLLON;
    while ((RCC->CR & RCC_CR_PLLRDY) == 0UL) { }
    RCC->CFGR = (RCC->CFGR & ~RCC_CFGR_SW) | RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) { }
}
else /* 无晶振退路 */
{
    RCC->CR |= RCC_CR_HSION;
    while ((RCC->CR & RCC_CR_HSIRDY) == 0UL) { }
    FLASH_ACR = FLASH_ACR_PRFTBE;
    RCC->CFGR &= ~RCC_CFGR_SW;
}
}

/* 由寄存器反推当前主频，保证延时不随晶振失效而变快 */
static uint32_t Clock_GetHclkHz(void)
{
    uint32_t sws = RCC->CFGR & RCC_CFGR_SWS;
    if (sws == RCC_CFGR_SWS_PLL)
    {
        uint32_t mul = ((RCC->CFGR >> 18) & 0x0FUL) + 2UL;
        uint32_t src = (RCC->CFGR & RCC_CFGR_PLLSRC_HSE)
            ? ((RCC->CFGR & RCC_CFGR_PLLXTPRE_HSE_DIV2) ? 4000000UL : 8000000UL)
            : 4000000UL;
        return src * mul;
    }
    return 8000000UL;
}

/* ===== 三、SysTick 毫秒延时(轮询，不开中断) ===== */
static void SysTick_DelayMs(uint32_t ms)
{
    uint32_t reload = (g_hclk_hz / 1000UL) - 1UL; /* 72MHz -> 71999 */
    while (ms--)

```

```

    {
        SysTick->LOAD = reload;
        SysTick->VAL = 0UL;
        SysTick->CTRL = SYSTICK_CTRL_CLKSOURCE | SYSTICK_CTRL_ENABLE;
        while ((SysTick->CTRL & SYSTICK_CTRL_COUNTFLAG) == 0UL) { }
        SysTick->CTRL = 0UL;
    }
}

/* ===== 四、GPIO 寄存器操作 ===== */
static void GPIO_ClockEnable(GPIO_TypeDef *port)
{
    if (port == GPIOA) { RCC->APB2ENR |= RCC_APB2ENR_IOPAEN; }
    else if (port == GPIOB) { RCC->APB2ENR |= RCC_APB2ENR_IOPBEN; }
    else if (port == GPIOC) { RCC->APB2ENR |= RCC_APB2ENR_IOPCEN; }
    (void)RCC->APB2ENR;
}

static void GPIO_ConfigOutput(GPIO_TypeDef *port, uint8_t pin, uint32_t mode_cnf)
{
    __IO uint32_t *cr;
    uint32_t shift;
    if (pin < 8U) { cr = &port->CRL; shift = (uint32_t)pin * 4UL; }
    else { cr = &port->CRH; shift = ((uint32_t)pin - 8UL) * 4UL; }
    *cr = (*cr & ~(0xFUL << shift)) | ((mode_cnf & 0xFUL) << shift);
}

static void GPIO_WritePin(GPIO_TypeDef *port, uint8_t pin, uint32_t level)
{
    if (level) { port->BSRR = (1UL << pin); } /* 置位: 输出高 */
    else { port->BSRR = (1UL << (pin + 16UL)); } /* 复位: 输出低 */
}

/* ===== 五、LED 驱动 ===== */
static void LED_On(const LED_t *led);
static void LED_Off(const LED_t *led);

static void LED_Init(void)
{
    uint8_t i;
    for (i = 0U; i < (uint8_t)(sizeof(s_leds) / sizeof(s_leds[0])); i++)
    {
        GPIO_ClockEnable(s_leds[i].port);
    }
}

```

```

        if (s_leds[i].port == GPIOC && s_leds[i].pin >= 13U)
            GPIO_ConfigOutput(s_leds[i].port, s_leds[i].pin, GPIO_OUT_PP_2MHZ);
        else
            GPIO_ConfigOutput(s_leds[i].port, s_leds[i].pin, GPIO_OUT_PP_50MHZ);
        LED_Off(&s_leds[i]);
    }
}

static void LED_On(const LED_t *led)
{
    GPIO_WritePin(led->port, led->pin, (led->active_low != 0U) ? 0UL : 1UL);
}

static void LED_Off(const LED_t *led)
{
    GPIO_WritePin(led->port, led->pin, (led->active_low != 0U) ? 1UL : 0UL);
}

/* ===== 六、主函数 ===== */
int main(void)
{
    uint8_t i;
    g_hclk_hz = Clock_GetHclkHz(); /* 记录主频 */
    LED_Init(); /* 配置 4 个 LED 引脚 */

    for (;;)
    {
        for (i = 0U; i < LED_NUM; i++) /* 轮流点亮每一只 LED */
        {
            LED_On(&s_leds[i]);
            SysTick_DelayMs(LED_ON_TIME_MS); /* 间隔 1 秒 */
            LED_Off(&s_leds[i]);
        }
    }
}

```

## 附录 C 参考资料

1. STMicroelectronics. \*RM0008 — STM32F10xxx 参考手册\* (第 9 章 GPIO、第 7 章 RCC、第 3 章 Flash)
2. STMicroelectronics. \*STM32F103x8/xB 数据手册\* (引脚定义与电气特性, PC13 的 3mA 限制)
3. STMicroelectronics. \*PM0056 — STM32F10xxx/20xxx/21xxx/L1xxxx Cortex-M3 编程手册\* (SysTick)

4. STM32 寄存器实现流水灯 — CSDN:  
<[https://blog.csdn.net/qq\\_47281915/article/details/120812867](https://blog.csdn.net/qq_47281915/article/details/120812867)>
  5. STM32F103C8T6 最小系统板原理图与引脚图 — CSDN:  
<[https://blog.csdn.net/m0\\_63323712/article/details/133362282](https://blog.csdn.net/m0_63323712/article/details/133362282)>
  6. 野火/正点原子 STM32 开发指南 (GPIO 与时钟树部分)
  7. Git 官方文档: <<https://git-scm.com/book/zh/v2>>
- 

**(完)**